

OOP

UNIT 4 – DESTRUCTORS

Every object has a lifetime. Constructors define how an object begins its life, while destructors define how it ends.

LEARNING OBJECTIVES

Explain why destructors exist.

Write a destructor using correct C++ syntax.

Describe when destructors execute.

Extend about object lifetime and scope.

Summarize about constructors and destructors.

Trace construction and destruction in objects that contain objects.

Explain why the current Song destructor is empty.

OBJECT DESTRUCTION

In the previous unit we learned how an object begins its life. A constructor initializes the object and prepares it for use.

```
Song song("Imagine", "John Lennon", 183);  
song.play();  
song.print();
```

Eventually the object reaches the end of its lifetime. When this happens, C++ destroys the object automatically.

The special member function responsible for the final stage of object lifetime is called the destructor.

OBJECT LIFETIME

Every object follows the same basic lifecycle:

Object Declaration	v
Constructor Executes	v
Object Ready	v
Member Functions	v
Destructor Executes	

The constructor marks the beginning of the lifetime. The destructor marks its end.

DEFINING A DESTRUCTOR

A destructor is a special member function that executes automatically when an object is destroyed.

```
class Song
{
public:
    ~Song()
    {
    }
};
```

The destructor has the same name as the class, preceded by the tilde character (~).

- It has no return type.
- It cannot receive parameters.
- It executes automatically.
- A class has only one destructor.

Like a constructor, a destructor may be defined inside the class or declared in the class and implemented in the source file.

DESTRUCTOR DEFINED OUTSIDE THE CLASS

```
#include <iostream>
using namespace std;
class Song {
public:
    Song();
    ~Song();
};
Song::Song() {
    cout << "Constructor" << endl;
}
Song::~~Song() {
    cout << "Destructor" << endl;
}
int main() {
    Song song;
    return 0;
}
```

Output:

```
Constructor
Destructor
```

The expression `Song::~~Song()` means that the destructor belongs to the `Song` class.

AUTOMATIC DESTRUCTOR EXECUTION

C++ calls destructors automatically when an object reaches the end of its lifetime.

```
#include <iostream>
using namespace std;
class Song{
public:
    Song() { cout << "Constructor" << endl; }
    ~Song() { cout << "Destructor" << endl; }
};
int main() {
    cout << "Before object" << endl;
    Song song;
    cout << "Object is working" << endl;
    return 0;
}
```

Output:

```
Before object
Constructor
Object is working
Destructor
```

WHY DESTRUCTORS EXIST

A destructor gives an object a final opportunity to clean up resources before it disappears. Some classes own memory, files, connections, or other resources that must not remain active after the object is gone.

Our current Song class does not yet own such resources, so its destructor may be empty. The destructor still executes.

Think of an object as a student in a classroom.

The **constructor** is like the student arriving and taking a seat.

During the lesson, the seat is occupied and cannot be used by anyone else.

When the lesson ends, the student leaves and frees the seat.

The **destructor** plays a similar role by allowing an object to release what it was using before it disappears.

WHEN EXACTLY IS A DESTRUCTOR CALLED?

For a **local** object, the destructor executes when execution leaves the scope in which the object was declared.

This may happen because a **function ends**, a **block ends**, or a **return** statement leaves the function.

SCOPE

```
#include <iostream>
using namespace std;

class Song
{
public:
    Song() { cout << "Song created" << endl; }
    ~Song() { cout << "Song destroyed" << endl; }
};

void playSong()
{
    Song song;
    cout << "Inside playSong" << endl;
}
```

```
int main() {  
    cout << "* Before call" << endl;  
    playSong();  
    cout << "* After call" << endl;  
    cout << "** Before block" << endl;  
    {  
        Song song;  
        cout << "Inside block" << endl;  
    }  
    cout << "** After block" << endl;  
}
```

Output:

```
* Before call  
Song created  
Inside playSong  
Song destroyed  
* After call  
  
** Before block  
Song created  
Inside block  
Song destroyed  
** After block
```


EARLY RETURN

```
void test(bool stop)
{
    Song song;
    if (stop)
        return; //when return all local objects are released
    cout << " didn't stop \n";
}
```

The destructor executes before the function returns. Automatic cleanup is connected to scope, not to reaching the final closing brace in a particular way.

MULTIPLE LOCAL OBJECTS

A scope may contain several objects. Their constructors execute in declaration order.

```
#include <iostream>
#include <string>
using namespace std;
class Song {
private:
    string title;
public:
    Song(string t)    {
        title = t;
```

```

        cout << "Creating " << title << endl;
    }
    ~Song()    {
        cout << "Destroying " << title << endl;
    }
};

int main()
{
    Song first("Imagine");
    Song second("Yesterday");
    Song third("Halo");

    cout << "All songs created" << endl;
}

```

Output:

```

Creating Imagine
Creating Yesterday
Creating Halo
All songs created
Destroying Halo
Destroying Yesterday
Destroying Imagine

```

A CLASS HAS ONLY ONE DESTRUCTOR

A class may have several constructors .

```
class Song
{
public:
    Song();
    Song(string title);
    Song(string title, string artist, int duration);
    ~Song();
};
```

A destructor cannot receive parameters. Therefore, destructors cannot be overloaded.

```
~Song();           // legal
~Song(string message); // illegal
```

Regardless of how a Song object was created, the same cleanup process is used when it is destroyed.

CONSTRUCTOR VS. DESTRUCTOR

Constructor

Executes when an object is created

Initializes the object

May receive parameters

May be overloaded

Destructor

Executes when an object is destroyed

Performs final cleanup

Cannot receive parameters

Cannot be overloaded

COMPOSITION: ALBUM CONTAINS SONG OBJECTS

One class may contain objects of another class. This relationship is called composition.

```
#include <iostream>
using namespace std;
class Song {
public:
    Song()    { cout << "Song constructor" << endl; }
    ~Song()   { cout << "Song destructor" << endl; }
};
class Album {
private:
    Song song;
public:
    Album()   { cout << "Album constructor" << endl; }
    ~Album()  { cout << "Album destructor" << endl; }
};

int main()
{
    Album album;
}
```

Output:

```
Song constructor  
Album constructor  
Album destructor  
Song destructor
```

The contained Song is constructed before the Album constructor body. During destruction, the Album destructor body executes first, followed by the Song destructor.

TWO SONGS INSIDE AN ALBUM

```
class Album  
{  
private:  
    Song firstSong;  
    Song secondSong;  
  
public:  
    Album()  
        : firstSong("Imagine"),  
          secondSong("Jealous Guy")  
    {  
        cout << "Creating album" << endl;  
    }  
}
```

```
    ~Album()  
    {  
        cout << "Destroying album" << endl;  
    }  
};
```

Output:

```
Creating song: Imagine  
Creating song: Jealous Guy  
Creating album  
Destroying album  
Destroying song: Jealous Guy  
Destroying song: Imagine
```

Contained objects are constructed in declaration order and destroyed in reverse declaration order.

DO NOT CALL THE DESTRUCTOR MANUALLY

```
Song song;  
song.~Song();    // do not do this for a normal local object
```

For ordinary local objects, allow C++ to execute the destructor automatically.

SUMMARY

Destructors complete the object lifecycle. A constructor initializes an object when it is created, while a destructor performs the final actions required when the object is destroyed.

- A destructor has the class name preceded by ~.
- A destructor has no return type and receives no parameters.
- A class has only one destructor.
- Destructors execute automatically.
- Scope determines the lifetime of local objects.
- Local objects are destroyed in reverse construction order.
- Contained objects are destroyed automatically.
- The current Song destructor is empty because no manual cleanup is required.

REVIEW QUESTIONS

- a. What is a destructor?
- b. When does a destructor execute?
- c. How is a destructor name formed? Does a destructor have return type?
- d. Can a destructor receive parameters?
- e. Why can a class have several constructors but only one destructor?
- f. What is the relationship between scope and object lifetime?
- g. When is a local object declared inside a block destroyed?
- h. Does a destructor execute after an early return?
- i. What is composition?
- j. If Album contains Song, which constructor executes first?
- k. If Album contains two Song objects, in what order are they created\destroyed?

LOOKING AHEAD

In this unit we completed the basic object lifecycle. We now understand how objects are created, initialized, used, and destroyed.

For the current Song class, the destructor is simple because the class does not manually manage external resources.

Later we will allow classes to manage dynamic memory. At that point, the destructor will become responsible for releasing resources owned by the object. This will prepare us for copy constructors, assignment operators, deep copying, and the Rule of Three.